

T11: The Legend of Tuna: Breath of Catnip

- Do this teamwork assignment with a partner.

Learning Objectives

- Work with classes, specifically compositions and inheritance
- Learn the basics of Pygame

How to Begin

- To begin, make a copy of this document by going to File >> Make a Copy...
- Share the copied document with all members of your team. You can share this document by hitting the blue button in the top right of the document, then entering the email address of all members in the bottom input field.
- Change the file name of this document to **username1, username2 - T11: The Legend of Tuna: Breath of Catnip** (for example, **heggens, wilborned - T11: The Legend of Tuna: Breath of Catnip**). To do this, click the label in the top left corner of your browser.
- Navigate to the [GitHub Classroom for T11](#).
- You will be asked to create a team:
 - One of you will create the team
 - The other(s) will join the team when prompted.
- Paste the link to your team's GitHub repository here:

GitHub Repository Link:	https://github.com/brymoreno01/likhachevai_morenocasasb
-------------------------	---

- Open PyCharm. If you are not at the Welcome to PyCharm page, go to File >> Close project.
- Using the **Clone Repository** button, open your repository using the URL above.
- In the PyCharm menu bar, go to **Git >> New Branch...**
- Name the branch your usernames (e.g., **heggens_wilborned**). Click **Create**.

Roles

Go to t11_questions.md and complete the SECTION 1 section, then come back to this document to complete the assignment.
--

Pygame

[Pygame](#) is a Python library built explicitly for constructing games. Why a whole library just for games? There are a slew of challenges that get introduced in the gaming industry. For example, how do we handle two objects colliding in a game? How do we handle two objects moving at the same time? What if we have 100 objects? What if we need gravity? Pygame helps us do many of these very challenging tasks by providing us with simple functions and methods for handling them.

In this assignment, we won't get to all of them (not even close!), but we hope this will give you a taste of what is possible for, say, a final project.

Run the **t11_game.py** file and play the game a few times to understand how it functions, then come back here to read and answer some questions about how it works.

Games and Object-Oriented Programming

Because games are often made up of real-world analogies, for example, a Player, an Enemy, an Inventory, object-oriented programming is an excellent way to structure your code. One class that is often forgotten about is the most obvious: a `Game` class. The `Game` class handles controlling all of the other components: creating the player, spawning enemies, handling changes between objects (such as adding coins to an inventory), and much, much more.

Go to **t11_questions.md** and complete the **SECTION 2**, then come back to this document to complete the assignment.

Inheritance

Inheritance provides us programmers with a means for extending and reusing code without having to duplicate it. Often, a class will inherit functionality from one other class (which may, in turn, inherit from another class, ...); we call the original class we are reusing the parent class, while the new class which is inheriting attributes and methods is called the child class. In our code, we are already doing this!

Go to **t11_questions.md** and complete the **SECTION 3**, then come back to this document to complete the assignment.

The Legend of Tuna: Breath of Catnip

Currently, our hero Tuna wanders the world in search of his one true love, Taco Cat. Taco Cat is a busy cat though, constantly searching the world for adventure. Taco Cat doesn't have time for Tuna. However, when Tuna finally catches Taco Cat, they settle down and enjoy a life together eating fish tacos.

It's time to extend our game.

The NPC class (NPC stands for non-player character) was implemented with the expectation that there would only be a "good" NPC (i.e., Taco Cat). But, we want to add a second NPC to the game now, an evil cat which goes by the name Whiskers. To implement our evil cat, we need to refactor NPC so that there are three classes in total:

- A parent class which holds attributes and methods that belongs to ALL NPCs
- A child "good" NPC class, which holds attributes and methods for good NPCs (i.e., Taco Cat).
- A child "evil" NPC class, which holds new attributes and methods for evil NPCs (yet to be implemented Whiskers).

Go to **t11_questions.md** and complete the **SECTION 4**, then come back to this document to complete the

assignment.

Final Thoughts

Go to **t11_questions.md** and complete the **SECTION 5**, then come back to this document to complete the assignment.

Submission Instructions

At the end of every assignment, I will include these instructions. They do change on occasion, so be sure you check them each assignment to ensure no special instructions were added.

Follow the [submission instructions](#) by Friday at 11:55PM.